

**EXPERIMENT NO. 7****Date:** \_\_\_\_\_**TITLE**

K-Nearest Neighbors Classification on Fashion MNIST

**AIM**To classify fashion images with K-Nearest Neighbors for  $K = 1$  to 15 and compare accuracy and voting time.**OBJECTIVE**

---

To implement K-Nearest Neighbors for image classification on Fashion MNIST, experiment with different values of  $K$ , and analyze their impact on accuracy and computation.

**THEORY**

Euclidean distance between test and train images:

$$d(x, z) = \sqrt{\sum_{j=1}^{784} (x_j - z_j)^2}$$

Majority vote of the  $K$  nearest neighbours:

$$\hat{y} = \arg \max_c \sum_{i \in N_K(x)} [y_i = c]$$

Test accuracy:

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n [\hat{y}_i = y_i]$$

Small  $K$  overfits noise; large  $K$  over-smooths; distance computation dominates runtime.**PROCEDURE**

1. Load Fashion MNIST ( $28 \times 28$  grey images, 10 classes); normalize pixels to  $[0, 1]$ .
2. Take a 6000/1000 train/test subset (seed 42); build the test-train distance matrix.
3. Implement majority-vote prediction for  $K = 1, 3, 5, 7, 10, 15$ .
4. Record accuracy and voting time per  $K$ ; plot accuracy vs  $K$  and discuss.

**DATASET DESCRIPTION**

Dataset Name: Fashion MNIST (IDX files, downloaded once)

Source: 60000 train + 10000 test images,  $28 \times 28$  grey, 10 classes

Number of Samples: 6000 train / 1000 test used    Number of Features: 784 pixels

Target / Class: Garment class (0–9)

Description: Pixels scaled to  $[0, 1]$ ; Euclidean distance; seed 42.

## PROGRAM

```

import os
import time
import gzip
import urllib.request
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
BASE = "http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/"
FILES = ["train-images-idx3-ubyte", "train-labels-idx1-ubyte",
         "t10k-images-idx3-ubyte", "t10k-labels-idx1-ubyte"]
HERE = os.path.dirname(os.path.abspath(__file__))
for f in FILES:
    p = os.path.join(HERE, f + ".gz")
    if not os.path.exists(p):
        print("Downloading", f)
        urllib.request.urlretrieve(BASE + f + ".gz", p)
def load_img(name, n):
    with gzip.open(os.path.join(HERE, name + ".gz"), "rb") as f:
        return np.frombuffer(f.read(), dtype=np.uint8, offset=16).reshape(n, 784).astype(
            np.float32) / 255.0
def load_lbl(name, n):
    with gzip.open(os.path.join(HERE, name + ".gz"), "rb") as f:
        return np.frombuffer(f.read(), dtype=np.uint8, offset=8)[:n]
X_all = load_img("train-images-idx3-ubyte", 60000)
y_all = load_lbl("train-labels-idx1-ubyte", 60000)
Xt_all = load_img("t10k-images-idx3-ubyte", 10000)
yt_all = load_lbl("t10k-labels-idx1-ubyte", 10000)
print("Full: train", X_all.shape, " test", Xt_all.shape)
rng = np.random.default_rng(42)
tri = rng.permutation(60000)[:6000]
tei = rng.permutation(10000)[:1000]
Xtr, ytr = X_all[tri], y_all[tri]
Xte, yte = Xt_all[tei], yt_all[tei]
print("Used: train", Xtr.shape, " test", Xte.shape)
t0 = time.perf_counter()
D = np.empty((len(Xte), len(Xtr)), dtype=np.float32)
for i in range(0, len(Xte), 100):
    D[i:i + 100] = ((Xte[i:i + 100, None, :] - Xtr[None, :, :]) ** 2).sum(-1)
print(f"Distance matrix {D.shape} in {time.perf_counter() - t0:.1f}s")
acc = []
for K in [1, 3, 5, 7, 10, 15]:
    t1 = time.perf_counter()
    nn = np.argsort(D, axis=1)[:K]
    votes = ytr[nn]
    pred = np.array([np.bincount(r, minlength=10).argmax() for r in votes])
    a = float((pred == yte).mean())
    acc.append(a)
    print(f"K = {K:2d}  accuracy = {a:.4f}  vote time = {time.perf_counter() - t1:.2f}s")
print(f"Best K = {[1, 3, 5, 7, 10, 15][int(np.argmax(acc))]} with accuracy {max(acc):.4f}")
plt.figure(figsize=(7, 4.5))
plt.plot([1, 3, 5, 7, 10, 15], acc, marker="o", linewidth=2)
plt.xlabel("K")
plt.ylabel("Test accuracy")
plt.title("KNN on Fashion MNIST: Accuracy vs K")
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(HERE, "exp7_acc.png"), dpi=150)

```

## OUTPUT

```
Full: train (60000, 784) Test: (10000, 784)
Used: train (6000, 784) test (1000, 784)
Distance matrix (1000, 6000) in 19.0s
K = 1 accuracy = 0.7890 vote time = 0.03s
K = 3 accuracy = 0.8010 vote time = 0.03s
K = 5 accuracy = 0.8040 vote time = 0.03s
K = 7 accuracy = 0.8080 vote time = 0.03s
K = 10 accuracy = 0.8130 vote time = 0.03s
K = 15 accuracy = 0.8030 vote time = 0.03s
Best K = 10 with accuracy 0.8130
```

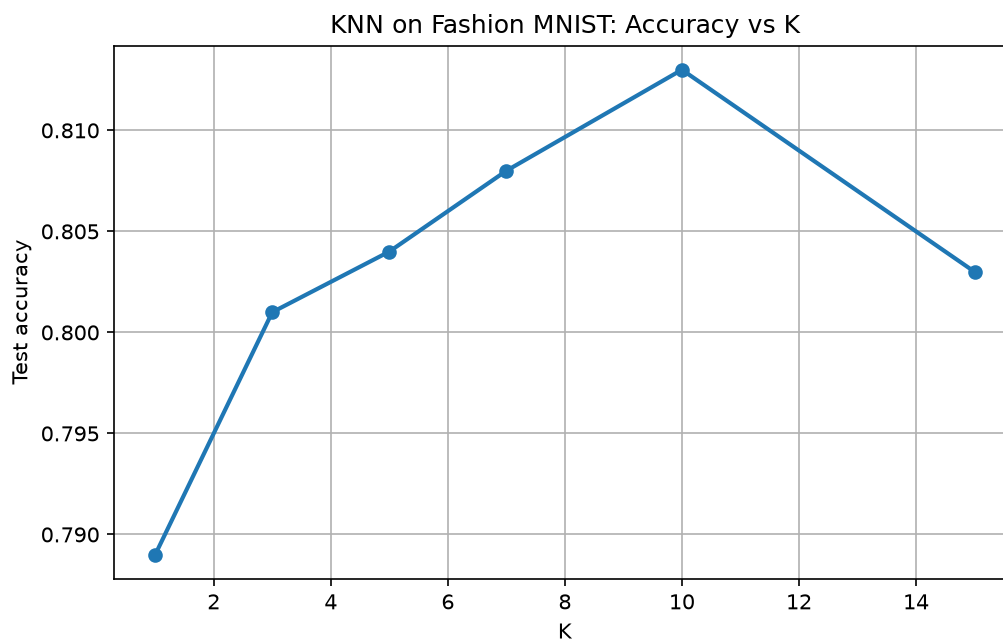


Fig. 1: Test accuracy vs  $K$  (peak at  $K = 10$ ).

## RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **K-Nearest Neighbors classification** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: \_\_\_\_\_

Date: \_\_\_\_\_